

Real-time communication with SignalR

Source code [here](#)

Demonstration video [here](#)

The ultimate goal of SignalR is to provide fast and simultaneous data transfers wherever required, like stock trading dashboards, online gaming as well as in real-time delivery service applications.

Let's learn about it, so engineers can get familiar with the capabilities of this exciting web technology and how to implement it with Blazor for a practical case, as it can be a chat room.

Author of the article: *Iker Elg. Alzaa*

#SignalR #Blazor #WebCommunication



SignalR is an ASP.NET Core library, that is used for real-time communication, as the client does not have to request new data in order for the data to be sent, it is sent when the data becomes available.

Usually, when it comes to real-time connections we can think also about WebSocket, which is a computer communications protocol providing full-duplex (two-way), persistent, and bidirectional communication channels over a single TCP connection.

Unlike HTTP, which is request-response based and closes after each interaction, WebSockets remain open, allowing servers to push real-time data to clients instantly.

So, why SignalR?

SignalR is an abstraction over some of the transports that are required for real-time communication between client and server.

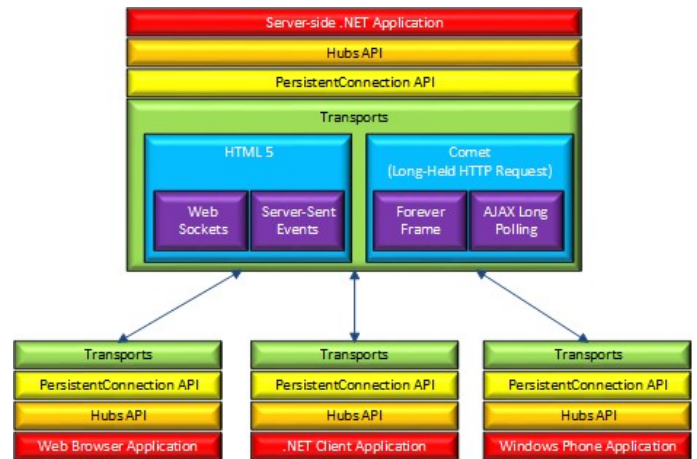
Automatically chooses the best transport that is available for both client and server. In fact, it will first attempt to establish a websocket connection, if that is possible, great, if not it will automatically fall back to the next best transport.

In this article we will describe how to implement this technology with a real-life example as it is a web chat room.

Architecture

At high level, a client-server SignalR chat where the Blazor UI acts as a SignalR client and communicates with a server to broadcast messages to all connected clients.

The following diagram shows the relationship between Hubs, persistent connections, and the underlying technologies used for transports.



Hubs

Hubs are an abstraction provided by SignalR that simplifies real-time communication by allowing methods to be called directly on the server from the client (and vice versa) without dealing with the underlying protocol.

Hubs take care of the complexity of dealing with connections, managing groups, and sending messages.

The next code example shows how to implement it with c# for a Blazor application. This class contains methods that clients can call.

```
BlazorChatSampleHub.cs
SignalRChatRoom
using Microsoft.AspNetCore.SignalR;
namespace WebApp.Hubs
{
    3 references | kraftcoding, 42 days ago | 1 author, 1 change
    public class BlazorChatSampleHub : Hub
    {
        public const string HubUrl = "/chatroom";

        0 references | kraftcoding, 42 days ago | 1 author, 1 change
        public async Task Broadcast(string username, string message)
        {
            await Clients.All.SendAsync("Broadcast", username, message);
        }

        0 references | kraftcoding, 42 days ago | 1 author, 1 change
        public override Task OnConnectedAsync()
        {
            Console.WriteLine($"*{Context.ConnectionId} connected*");
            return base.OnConnectedAsync();
        }

        0 references | kraftcoding, 42 days ago | 1 author, 1 change
        public override async Task OnDisconnectedAsync(Exception e)
        {
            Console.WriteLine($"*Disconnected {e?.Message} {Context.ConnectionId}*");
            await base.OnDisconnectedAsync(e);
        }
    }
}
```

Hubs are transient, every time a client connects to a Hub or calls methods, a new instance of Hub is created, they don't store data between these instances.

Hub class has two useful properties:

- Clients, allows sending messages to specific clients or groups of clients.
- Context, provides information about the current connection, such as connection ID, user identity, and HTTP context.

Blazor Application

A Blazor SignalR chat client UI connects to a server hub, sends/receives messages, and shows join/leave notices. This is what "SignalRChatRoom/Pages/ChatRoom.razor" does (see code).

UI flow:

- When not chatting it asks for a username (input is currently "disabled"), shows a "Chat" button.
- When chatting it shows a banner with the current user, a "Disconnect" button, the message list, a textarea for new messages and a "Send" button.

Connection lifecycle:

- "Chat()" validates username, sets "_isChatting", builds a "HubConnection" to "navigationManager.BaseUri +

BlazorChatSampleHub.HubUrl", registers a "Broadcast" handler, starts the connection, clears old messages and sends a join notice.

- "DisconnectAsync()" sends a leave notice, stops and disposes the hub connection, clears state.

Messaging:

- "_hubConnection.On<string,string>('Broadcast', BroadcastMessage)" routes incoming messages to "BroadcastMessage", which adds a "Message" to "_messages" and triggers UI update via "InvokeAsync(StateHasChanged)".
- "SendAsync(string message)" sends "Broadcast" with "_username" and the message when "_isChatting".

Data model:

- Inner "Message" class includes "Username", "Body", "Mine", "IsNotice" (checks "[Notice]" prefix) and "CSS" for sent/received styling.